

Phased Implementation Plan & Roadmap

Revision: 1 Last modified: 2026-07-24T00:00:00Z

Phase : Foundation (Weeks –)

Goal: Establish the LLM infrastructure, Constitution integration, MCP deployment, and SpecKit installation so all subsequent phases build on a solid foundation.

Week — Environment & LLM Setup

Day	Task	Owner	Deliverable
Mon	Provision Helix LLM API access — obtain API key, endpoint, rate limits	Ops	API credentials in <code>.env</code>
Mon	Install <code>node >=18</code> , <code>bash >=5</code> , <code>jq</code> , <code>ripgrep</code> , <code>graphviz</code> on all hosts	Ops	Verified <code>command -v</code> for each tool
Tue	Clone <code>helix-nano-bridge</code> template repo, scaffold directory structure	Dev	<code>tree constitution/submodules/helix-nano-bridge/</code>
Tue	Create <code>constitution/submodules/helix-nano-bridge/helix-deps.yaml</code> per §11.4.28(C)	Dev	Valid YAML, parsed by <code>constitution/scripts/check_deps.sh</code>
Wed	Configure Helix LLM fallback chain: <code>helix-nano</code> → <code>deepseek-v4-pro</code> → <code>opus-xhigh</code>	Dev	<code>.env.template</code> with all 3 endpoints
Wed	Write <code>bin/llm_probe.sh</code> — health-check script that tests all 3 LLM backends	Dev	Exit 0 = all backends reachable
Thu	Write <code>lib/llm_client.sh</code> — unified client calling Helix API with retry + backoff	Dev	Unit test: 3 consecutive successful calls
Fri	Integration test: <code>llm_probe.sh</code> → all backends GREEN, fallback chain verified	QA	Test log in <code>qa-results/week1/</code>
Fri	Week 1 retrospective + <code>docs/CONTINUATION.md</code> update	All	Updated §3 Active work

Week – Constitution Integration

Day	Task	Owner	Deliverable
Mon	Study <code>constitution/Constitution.md</code> §11.4.27 (HelixQA), §11.4.43 (TDD), §11.4.115 (RED/GREEN)	Dev	Notes in <code>docs/research/constitution_audit.md</code>
Tue	Map every Speckit extension hook to the corresponding Constitution §-letter	Dev	Traceability matrix <code>docs/research/speckit_constitution_map.md</code>
Tue	Write <code>CM-HELIX-NANO-BRIDGE-EXTENSION-EXISTS</code> pre-build gate	Dev	Gate in <code>pre_build_verification.sh</code>
Wed	Write paired §1.1 meta-test mutation for the gate	Dev	Mutation in <code>meta_test_false_positive_proof.sh</code>
Thu	Wire <code>helix-nano-bridge</code> into <code>docs_chain</code> per §11.4.106	Dev	<code>.docs_chain/contexts/helix_nano_bridge.yaml</code>
Fri	Verify all 5 Constitution <code>CM-COVENANT - *</code> propagation gates still GREEN after extension	QA	<code>pre_build_verification.sh</code> exit 0, 0 WARN
Fri	Week 2 retrospective	All	CONTINUATION.md updated

Week — MCP Server Deployment

Day	Task	Owner	Deliverable
Mon	Scaffold <code>superbridge-mcp</code> TypeScript project: <code>package.json</code> , <code>tsconfig.json</code> , <code>src/index.ts</code>	Dev	<code>npm install</code> && <code>npm run build</code> succeeds
Tue	Implement <code>execute_tdd_cycle</code> tool with RED → GREEN → VERIFY pipeline	Dev	Integration test: mocked TDD cycle exits correctly
Wed	Implement <code>verify_anti_bluff</code> tool — evidence file parsing + pattern matching	Dev	Unit tests for golden-good / golden-bad inputs
Thu	Implement <code>nano_task_decompose</code> + <code>nano_task_execute</code> + <code>nano_task_graph</code> tools	Dev	5 tools registered, each with input validation
Fri	Wire MCP server into Claude Code, Co-dex, Cursor, Gemini CLI configs	Dev	<code>mcp_server_test.sh</code> probes each agent
Fri	Week 3 retrospective	All	CONTINUATION.md updated

Week – SpecKit Installation & Smoke Test

Day	Task	Owner	Deliverable
Mon	Install SpecKit CLI: <code>specify --version</code> resolves	Dev	<code>specify --version ≥ 2.0.0</code>
Tue	Install SpecKit extensions: <code>superb</code> , <code>helix-nano-bridge</code> (skeleton)	Dev	<code>specify extension list</code> shows both
Tue	Create skeleton <code>extension.yml</code> with all 5 commands registered	Dev	<code>specify helix-nano-bridge --help</code> prints usage
Wed	Write <code>speckit.helix-nano-bridge.decompose.md</code> command spec	Dev	Markdown file with full spec per EXTENSION_DEVELOPMENT.md
Thu	Write remaining 4 command specs: <code>execute.md</code> , <code>graph.md</code> , <code>status.md</code> , <code>retry.md</code>	Dev	All 5 command specs complete
Fri	End-to-end smoke test: <code>specify feature new test-feature → decompose → execute</code> (dry-run)	QA	<code>qa-results/week4/smoke_test.log</code>
Fri	Phase 1 milestone review — foundation DECLARED READY	All	CM-HELIX-NANO-BRIDGE-EXTENSION-EXISTS GREEN

Phase Deliverables

```
constitution/submodules/helix-nano-bridge/
├─ bin/
|   └─ llm_probe.sh                # LLM health check
|   └─ helix-nano-bridge           # Main entry point (skeleton)
├─ lib/
|   └─ llm_client.sh              # Unified LLM client
├─ spec/
|   └─ helix-nano-bridge/
|       └─ extension.yml           # Speckit extension manifest
|       └─ speckit.helix-nano-bridge.decompose.md
|       └─ speckit.helix-nano-bridge.execute.md
|       └─ speckit.helix-nano-bridge.graph.md
|       └─ speckit.helix-nano-bridge.status.md
|       └─ speckit.helix-nano-bridge.retry.md
├─ mcp/
|   └─ superbridge-mcp/
|       └─ package.json
|       └─ tsconfig.json
|       └─ src/index.ts            # 5-tool MCP server
|       └─ dist/index.js          # Built artifact
├─ helix-deps.yaml                # §11.4.28(C) dependency spec
└─ .env.template
```

2

5

8

Phase : Nano-Task Engine (Weeks –)

Goal: Build the nano-task decomposition engine, DAG dependency resolver, TDD execution pipeline, and anti-bluff oracle — the core intellectual property.

Week — Nano-Task Schema & Decompose Command

Day	Task	Owner	Deliverable
Mon	Define <code>nano-tasks.json</code> JSON Schema (JSON Schema draft-2020-12)	Dev	<code>spec/nano-tasks.schema.json</code> — validates with <code>ajv</code>
Tue	Implement <code>nano_decompose_task()</code> in <code>lib/nano_bridge_common.sh</code>	Dev	Parses <code>tasks.md</code> → extracts files/symbols
Wed	Implement <code>nano_build_dependency_dag()</code> — adjacency list from <code>plan.md</code>	Dev	DAG builder with test coverage
Thu	Implement <code>nano_detect_cycles()</code> — Kahn's algorithm with DFS fallback	Dev	Cycle detection test: inject cycle → detected
Fri	Wire <code>helix-nano-bridge decompose</code> command end-to-end	Dev	<code>nano-tasks.json</code> produced from real <code>tasks.md</code>
Fri	Week 5 retrospective	All	CONTINUATION.md updated

Week — DAG Engine & Phase Assignment

Day	Task	Owner	Deliverable
Mon	Implement <code>nano_assign_phases()</code> — level-order grouping	Dev	Tasks assigned to phases, no file collisions in same phase
Tue	Implement critical path computation — longest path in DAG	Dev	<code>nano-tasks.json</code> includes <code>critical_path_length</code>
Wed	Implement <code>nano_generate_mermaid()</code> / <code>nano_generate_dot()</code>	Dev	Mermaid and DOT output renderable in tools
Thu	Wire <code>helix-nano-bridge</code> graph command	Dev	<code>--format mermaid</code> → valid chart, <code>--format svg</code> → renders
Fri	Test DAG correctness on 10 sample <code>tasks.md</code> files (varying complexity)	QA	All 10 produce acyclic, correctly-phased DAGs
Fri	Week 6 retrospective	All	CONTINUATION.md updated

Week — TDD Execution Engine

Day	Task	Owner	Deliverable
Mon	Implement <code>nano_execute_single_task()</code> — RED → GREEN → VERIFY cycle	Dev	Core execution loop
Tue	Implement RED phase: run test, assert FAIL, capture evidence	Dev	<code>RED_<timestamp>.log</code> with exit code $\neq 0$
Wed	Implement GREEN phase: run test, assert PASS, 3-iteration consistency	Dev	<code>GREEN_<timestamp>.log</code> with exit code = 0, all 3 identical
Thu	Implement VERIFY phase: anti-bluff oracle checks evidence files	Dev	<code>verdict.json</code> with PASS/FAIL + rationale
Fri	Wire <code>helix-nano-bridge execute</code> command — phase-by-phase execution	Dev	Phases execute in order, parallel within phase
Fri	Week 7 retrospective	All	CONTINUATION.md updated

Week — Unit Tests & Anti-Bluff Oracle

Day	Task	Owner	Deliverable
Mon	Write unit tests for <code>nano_decompose_task()</code> — 10+ test cases	Dev	100% pass rate
Tue	Write unit tests for <code>nano_build_dependency_dag()</code> — cycle/no-cycle/multi-dep	Dev	100% pass rate
Wed	Write unit tests for <code>nano_execute_single_task()</code> — RED-fail/GREEN-pass/verify	Dev	100% pass rate
Thu	Implement anti-bluff oracle golden-good / golden-bad fixtures	Dev	Golden-good PASSES oracle, golden-bad FAILS it
Fri	Wire paired §1.1 mutation for the execution gate	Dev	Stripping gate check → gate FAILS
Fri	Phase 2 milestone review — nano-task engine DECLARED READY	All	CM-NANO-TASK-ENGINE GREEN

Phase

Deliverables

lib/	
└─ nano_bridge_common.sh	# Full implementation (~600 lines)
└─ anti_bluff_oracle.sh	# Evidence validation oracle
└─ test_nano_bridge.sh	# Unit test suite for library
└─ test_anti_bluff_oracle.sh	# Oracle golden-good/golden-bad
tests	
bin/	
└─ helix-nano-bridge-decompose	# Full implementation
└─ helix-nano-bridge-execute	# Full implementation
└─ helix-nano-bridge-graph	# Full implementation
└─ helix-nano-bridge-status	# Full implementation
└─ helix-nano-bridge-retry	# Full implementation
spec/	
└─ nano-tasks.schema.json	# JSON Schema for nano-tasks.json
qa-results/week8/	
└─ unit_tests.log	# All unit tests GREEN
└─ decompose_10_samples.log	# 10/10 DAGs correct
└─ oracle_golden_fixtures.log	# Good=PASS, Bad=FAIL

3

9 1 2

Phase

: Superbridge Integration

(Weeks

–

)

Goal: Build the MCP server, SuperB extension integration, and E2E tests so the full pipeline works from `tasks.md` to verified implementation.

Week — MCP Server Completion

Day	Task	Owner	Deliverable
Mon	Complete <code>execute_tdd_cycle</code> tool — full RED → GREEN → VERIFY with evidence capture	Dev	Tool returns structured PASS/FAIL verdict
Tue	Complete <code>verify_anti_bluff</code> tool — expected/forbidden pattern matching	Dev	Tool catches golden-bad inputs
Wed	Complete <code>nano_task_decompose</code> tool — calls bash library via <code>exec</code>	Dev	Tool produces valid <code>nano-tasks.json</code>
Thu	Complete <code>nano_task_execute</code> + <code>nano_task_graph</code> tools	Dev	All 5 tools functional
Fri	MCP server integration test: Claude Code calls <code>execute_tdd_cycle</code> on a mock task	QA	PASS verdict with evidence JSON
Fri	Week 9 retrospective	All	CONTINUATION.md updated

Week – SuperB Integration

Day	Task	Owner	Deliverable
Mon	Map <code>superb</code> 7-command workflow to <code>helix-nano-bridge</code> 5 commands	Dev	<code>superb/integrations/helix-nano-bridge.yaml</code>
Tue	Wire <code>superb:plan</code> → <code>decompose</code> , <code>superb:implement</code> → <code>execute</code>	Dev	SpecKit hooks fire correctly
Wed	Implement <code>implementation-gate.js</code> — 7 checks before execution	Dev	<code>node implementation-gate.js <feature_dir></code>
Thu	Wire <code>superb:verify</code> → <code>status --format report --evidence</code>	Dev	SuperB verify phase shows nano-task status
Fri	Wire <code>superb:ship</code> gated on 100% GREEN nano-tasks	Dev	Ship blocked when any task FAIL
Fri	Week 10 retrospective	All	CONTINUATION.md updated

Week – E E Tests

Day	Task	Owner	Deliverable
Mon	Create test feature: specify feature new e2e-nano-test with 3 tasks	Dev	tasks.md with 3 tasks, 5 dependencies
Tue	Run full pipeline: tasks.md → decompose → execute → graph → status	QA	100% GREEN, evidence captured for all nano-tasks
Wed	Inject a bug: modify a RED test to pass when it should fail → verify gate BLOCKS	QA	Gate exit 2, FAIL verdict recorded
Thu	Test retry: force-fail 2 tasks → retry → tasks recover and go GREEN	QA	Retry log shows RED → GREEN → VERIFY cycles
Fri	Test parallel execution: 24 independent nano-tasks → all run concurrently within phase	QA	Phase execution time ≤ sum of longest 2 tasks
Fri	Week 11 retrospective	All	CONTINUATION.md updated

Week — Completion & Polish

Day	Task	Owner	Deliverable
Mon	Write user documentation: docs/guides/ HELIX_NANO_BRIDGE_USER_GUIDE.md	Dev	Complete user guide, exported to HTML+PDF+DOCX
Tue	Write installation guide: docs/guides/ HELIX_NANO_BRIDGE_INSTALL.md	Dev	Step-by-step install, verified on fresh clone
Wed	Polish error messages — every exit code has a human-readable explanation	Dev	helix-nano-bridge --help shows all exit codes
Thu	Add helix-nano-bridge to constitu- tion/scripts/post_update_hook.sh per §11.4.164	Dev	Auto-registered on constitution pull
Fri	Phase 3 milestone review — Superbridge integration DECLARED READY	All	Full E2E test log GREEN
Fri	Week 12 retrospective	All	CONTINUATION.md updated

Phase

Deliverables

mcp/superbridge-mcp/	
├─ src/	
└─ index.ts	# 5-tool MCP server (complete)
└─ tdd_cycle.ts	# execute_tdd_cycle implementation
└─ anti_bluff.ts	# verify_anti_bluff implementation
└─ decompose.ts	# nano_task_decompose
implementation	
└─ execute.ts	# nano_task_execute implementation
└─ graph.ts	# nano_task_graph implementation
├─ tests/	
└─ tdd_cycle.test.ts	
└─ anti_bluff.test.ts	
└─ decompose.test.ts	
└─ e2e.test.ts	
├─ dist/	# Built JS output
└─ package.json	
superb/integrations/	
└─ helix-nano-bridge.yaml	# SuperB 7-command wiring
bin/	
└─ implementation-gate.js	# Pre-execution gate (7 checks)
docs/guides/	
├─ HELIX_NANO_BRIDGE_USER_GUIDE.md	
└─ HELIX_NANO_BRIDGE_INSTALL.md	
qa-results/week11/	
├─ e2e_full_pipeline.log	# GREEN
├─ gate_blocks_bad_test.log	# Gate caught the injected bug
├─ retry_recovery.log	# Tasks recovered
└─ parallel_benchmark.log	# Performance data

4

1316

Phase

: Production Hardening (Weeks

-)

Goal: Multi-host deployment, 100% test coverage, performance benchmarking, security audit, and documentation finalization — ready for production use.

Week — Multi-Host RPC & Scalability

Day	Task	Owner	Deliverable
Mon	Implement <code>superbridge-mcp</code> over HTTP/SSE transport (beyond stdio)	Dev	MCP server listens on TCP port, SSE streaming
Tue	Implement remote execution: nano-tasks dispatched to worker nodes	Dev	Task dispatched via <code>POST /execute</code> to remote MCP server
Wed	Implement distributed evidence store: shared <code>qa-results/</code> over NFS/SSHFS	Dev	All hosts read/write same evidence directory
Thu	Implement leader election for phase coordination — only one host advances phases	Dev	Leader lock via <code>flock</code> on shared NFS
Fri	Load test: 4 worker nodes, 200 nano-tasks, measure throughput	QA	Throughput report in <code>qa-results/week13/parallel_benchmark.md</code>
Fri	Week 13 retrospective	All	CONTINUATION.md updated

Week — % Test Coverage

Day	Task	Owner	Deliverable
Mon	Audit current test coverage — identify gaps	QA	Coverage report: <code>coverage_report.json</code>
Tue	Write tests for all error paths in <code>nano_bridge_common.sh</code>	Dev	Every <code>exit 1/2/3/4</code> path has a test
Wed	Write tests for all 5 MCP tools with edge cases	Dev	<code>npm test</code> → 100% pass, 80%+ coverage
Thu	Write chaos tests per §11.4.85: process death, disk full, OOM injection	Dev	Chaos tests in <code>tests/chaos/</code>
Fri	Write stress tests per §11.4.85: 100 concurrent decomposes, 1000-task DAG	QA	Stress report in <code>qa-results/week14/stress_tests.md</code>
Fri	Week 14 retrospective	All	CONTINUATION.md updated

Week — Performance Benchmarking & Security Audit

Day	Task	Owner	Deliverable
Mon	Benchmark decompose: 50-task <code>tasks.md</code> → nano-tasks (target: <5s)	QA	Latency p50/p95/p99 in benchmark report
Tue	Benchmark execute: 50 nano-tasks sequential (target: <2 min)	QA	Per-task latency distribution
Wed	Benchmark graph: 500-node DAG → Mermaid/DOT/SVG (target: <1s)	QA	Format-specific rendering times
Thu	Security audit: MCP server input validation, injection vectors, credential handling	Sec	Audit report in <code>docs/audit/helix_nano_bridge_security.md</code>
Fri	Fix all HIGH/MEDIUM security findings	Dev	Re-audit: 0 HIGH, 0 MEDIUM findings
Fri	Week 15 retrospective	All	CONTINUATION.md updated

Week

– Documentation & Release

Day	Task	Owner	Deliverable
Mon	Finalize <code>EXTENSION_DEVELOPMENT.md</code> — ensure every section complete	Dev	Document passes <code>CM-DOC-REVISION-HEADER-PRESENT</code>
Tue	Finalize <code>IMPLEMENTATION_PLAN.md</code> — verify all weeks mapped to actual deliverables	Dev	Document passes <code>CM-DOC-REVISION-HEADER-PRESENT</code>
Wed	Generate all 4 export formats (MD+HTML+PDF+DOCX) per §11.4.65	Dev	All mtimes $\geq$ source <code>.md</code>
Thu	Run full Constitution compliance sweep — all <code>CM-*</code> gates GREEN	QA	<code>pre_build_verification.sh</code> exit 0, 0 FAIL, 0 WARN
Fri	Tag <code>constitution</code> Submodule + create release <code>helix-nano-bridge-v1.0.0</code>	Ops	Tag on all remotes, <code>pack.sh</code> archive published
Fri	Phase 4 milestone review — PRODUCTION READY declared	All	All 4 phases complete

Phase Deliverables

```
mcp/superbridge-mcp/
├─ src/
│   └─ transport/
│       ├── stdio.ts           # Existing stdio transport
│       └─ http_sse.ts        # New HTTP/SSE transport
├─ tests/
│   ├── chaos/
│   │   ├── process_death.test.ts
│   │   ├── disk_full.test.ts
│   │   └─ oom_injection.test.ts
│   └─ stress/
│       ├── concurrent_decompose.test.ts
│       └─ large_dag.test.ts
└─ coverage/                  # Istanbul coverage reports

docs/
├─ audit/
│   └─ helix_nano_bridge_security.md # Security audit findings
├─ benchmarks/
│   └─ performance_2026Q3.md        # Latency + throughput data
└─ changelogs/
    └─ v1.0.0.md                   # Release changelog

dist/
└─ helix-nano-bridge-1.0.0.tar.gz  # Packaged release
```

Milestones & Deliverables

Mile-stone	End of Phase	Deliverables	Evidence
M1: Founda-tion	Week 4	LLM configured, SpecKit installed, MCP skeleton running, 5 com-mand specs com-plete	<code>llm_probe.sh</code> GREEN, <code>specify helix-nano-bridge --help</code> works, MCP server connects to Claude Code
M2: Nano-Task En-gine	Week 8	Decompose, ex-ecute, graph, status, retry all functional; DAG engine correct; anti-bluff oracle validated	All 5 commands tested on real <code>tasks.md</code> , <code>nano-tasks.json</code> valid, golden-good/bad fixtures pass
M3: Su-per-bridge Integra-tion	Week 12	MCP server com-plete (5 tools), SuperB integ-rated, E2E pipeline GREEN, user docs pub-lished	Full E2E log: decompose → execute → graph → status all GREEN
M4: Pro-duction Ready	Week 16	Multi-host, 100% coverage, secur-ity audited, per-formance bench-marked, docu-mented, tagged	<code>pre_build_verification.sh</code> 0 FAIL 0 WARN, release tag pushed, <code>pack.sh</code> archive with SHA256

Risk Register

#	Risk	Prob- abil- ity	Im- pact	Mitigation	Own- er
R1	Helix LLM API rate-limits during de-compose phase	Medi- um	High	Fallback to deepseek-v4-pro; cache decompositions; batch LLM requests	Dev Lead
R2	Dependency cycles in complex <code>tasks.md</code> files	Medi- um	Me- dium	Kahn's algorithm + DFS fallback in <code>nano_detect_cycles()</code> ; operator-override flag	Dev
R3	MCP server stdio transport incompatible with some agents	Medi- um	High	Ship both stdio and HTTP/SSE transports; test on all 4 agents in Week 3	Dev
R4	Evidence directory fills disk during long execution runs	Low	High	Per-task evidence size budget; auto-prune on GREEN; disk-space gate before execution	Ops
R5	Phase assignment creates false sharing (two tasks modify same file)	Medi- um	Me- dium	Phase collision detection in <code>nano_assign_phases()</code> ; implementation-gate.js check #7	Dev
R6	Constitution amendment invalidates SpecKit hook mapping	Low	High	<code>CM-HELIX-NANO-BRIDGE-EXTENSION-EXISTS</code> gate catches drift; <code>constitution/scripts/post_update_hook.sh</code> auto-revalidates	Dev Lead

#	Risk	Prob- abil- ity	Im- pact	Mitigation	Own- er
R7	Threadripper NUMA topology causes uneven task distribution	Medium	Low	<code>numactl --cpunodebind</code> per-worker; hardware detection auto-configures	Dev
R8	Speckit API breaks backward compatibility (v3.x)	Low	Medium	Extension manifest declares <code>speckitVersion: ">=2.0.0"</code> ; pin version in CI	Dev
R9	Security vulnerability in MCP server input handling	Medium	Critical	Input validation on all 5 tools; security audit Phase 4 Week 15; never eval user input	Sec
R10	Operator time unavailable for milestones (blocked decisions)	Medium	High	Per §11.4.66 interactive clarification; autonomous decisions on low-blast-radius items per §11.4.101	PM

Resource Requirements

Hardware (per development host)

Component	Minimum	Recommended	Purpose
CPU	16 cores	32+ cores (Threadripper)	Parallel nano-task execution
RAM	32 GB	64 GB	LLM context windows + parallel workers
GPU	None	NVIDIA RTX 4090 / A6000	LLM inference offload
Storage	50 GB free	200 GB NVMe	Evidence artifacts, build artifacts
Network	1 Gbps	10 Gbps	Multi-host coordination, evidence sync

Software

Tool	Version	Purpose
Node.js	≥18 LTS	MCP server runtime
Bash	≥5.0	Core scripting
jq	≥1.6	JSON processing
ripgrep	≥13	Fast file search
Graphviz	≥2.50	DAG rendering (DOT/SVG/PNG)
TypeScript	≥5.0	MCP server source
SpecKit CLI	≥2.0.0	Extension host
Superpowers CLI	≥3.1.0	Skill host

Personnel

Role	Count	Commitment
Lead Developer	1	Full-time, 16 weeks
Developer (MCP + JS)	1	Full-time, weeks 3–15
QA Engineer	1	50%, weeks 4–16
Security Auditor	1	2 weeks, weeks 15
Operator/PM	1	Advisory, milestone gates

Success Criteria (Anti-Bluff Evidence)

Each phase is complete ONLY when the following evidence is captured:

Phase Success Criteria

#	Criterion	Evidence Type
P1.1	<code>llm_probe.sh</code> exits 0 against all 3 LLM backends	Captured stdout with per-backend latency
P1.2	<code>specify helix-nano-bridge --help</code> prints usage for all 5 commands	Captured terminal output
P1.3	MCP server connects to Claude Code — <code>superbridge-mcp</code> listed in <code>ListTools</code>	MCP handshake log
P1.4	<code>CM-HELIX-NANO-BRIDGE-EXTENSION-EXISTS</code> gate GREEN	<code>pre_build_verification.sh</code> log
P1.5	Paired §1.1 mutation FAILs the gate	<code>meta_test_false_positive_proof.sh</code> log

Phase Success Criteria

#	Criterion	Evidence Type
P2.1	<code>decompose</code> produces valid <code>nano-tasks.json</code> from 10 diverse <code>tasks.md</code> files	10 <code>nano-tasks.json</code> files, all passing <code>ajv</code> validation
P2.2	Dependency graph is acyclic for all 10 samples	Cycle detection: 0 cycles found
P2.3	<code>execute</code> runs a full TDD cycle: RED fails → GREEN passes → VERIFY confirms	<code>verdict.json</code> with <code>PASS</code> and 3 iterations
P2.4	Anti-bluff oracle: golden-good → PASS, golden-bad → FAIL	Oracle test log
P2.5	All unit tests GREEN for library functions	Test runner output, 0 failures

Phase Success Criteria

#	Criterion	Evidence Type
P3.1	Full E2E: <code>tasks.md</code> (3 tasks) → de-compose → execute → all GREEN	<code>qa-results/e2e/full_pipeline.log</code>
P3.2	<code>implementation-gate.js</code> BLOCKS execution when RED test is missing	Gate exit 2, <code>FAIL</code> verdict
P3.3	Retry recovers intentionally-failed tasks	Retry log: RED → GREEN → VERIFY for each recovered task
P3.4	SuperB <code>superb:ship</code> blocked when nano-tasks not all GREEN	Ship gate log
P3.5	User guide complete: HTML+PDF+DOCX exports pass <code>CM-DOCS-EXPORT-SYNC</code>	Export freshness gate GREEN

Phase Success Criteria

#	Criterion	Evidence Type
P4.1	4 worker nodes execute 200 nano-tasks in parallel, throughput ≥ 10 tasks/s	Benchmark report in <code>qa-results/benchmarks/</code>
P4.2	Test coverage $\geq 85\%$ line, $\geq 80\%$ branch	Istanbul/NYC coverage report
P4.3	Chaos tests: process death, disk full, OOM $\rightarrow$ graceful degradation (no crash)	Chaos test logs with recovery traces
P4.4	Security audit: 0 HIGH, 0 MEDIUM findings	<code>docs/audit/helix_nano_bridge_security.md</code>
P4.5	<code>pre_build_verification.sh</code> exit 0, 0 FAIL, 0 WARN on release tag	Full pre-build sweep log
P4.6	Release tag <code>helix-nano-bridge-v1.0.0</code> pushed to all remotes per §11.4.113	<code>git ls-remote --tags</code> shows tag on all remotes
P4.7	<code>pack.sh</code> archive SHA256 published and verified	Release manifest <code>release-1.0.0.json</code>